Diamond Doc

A Software Documentation

Seepick

Version SNAPSHOT, 2025-12-07

Table of Contents

If you want to go fast and far in a sustainable way, you need to go slow and lightweight with your friends.

This document was built using AsciiDoc, a simple yet powerful markup language for writing technical documentation.

The build for this document was triggered on Sun, 7. December 2025 at 22:25 - enjoy :)

Figure 1. A precious material

About:

This project is a state-of-the-art prototype of a backend service, which can be used as a laboratory to gain experience (elaborating further on this proof-of-concept), a base for technical decision making and a template for future reference.

This document describes not only the specifics of this implementation (a tangible, hands-on, experience-based source of information using a concrete example instead of hypothetical theorization), but also the underlying principles, representing best practices. It is tailored to a technical prototype, thus there will be little to no mentioning of what could be expected for a regular service with business features and production relevance. Think of business concepts and rules, IT system landscape, API documentation, and much on the operational side (DevOps/CICD, releasing, performance, tracing, monitoring, alerting). It ends with an outlook on open doings and a future vision on how to shape also the broader picture of integration, deployment, monitoring, etc.

Decision Documentation:

We use ADRs (Architecture Decision Records) to document important architectural and technical decisions made during the development of this software project.

For more explanation what ADRs are, please read: https://github.com/seepick/diamond/blob/main/doc/decisions/README.adoc

Chapter Overview:

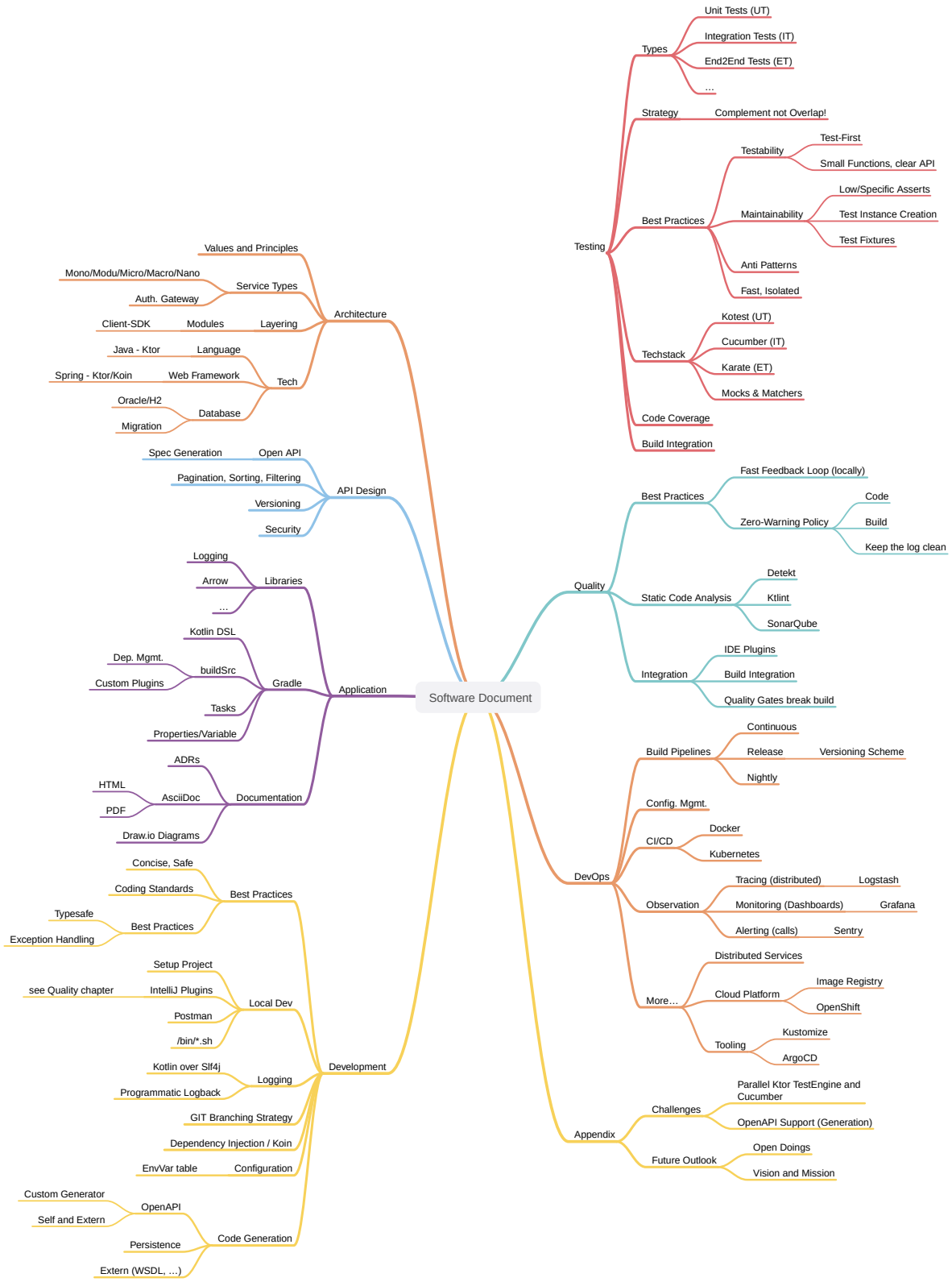

Figure 2. Sitemap of the book's content

Chapter 1. Architecture

1.1. Overall Guiding Principles

Based on those KPIs:

- Lead time of X days until feature available.
- High quality, many tests, low bugs.
- API response times of Xms (performance tests).
- Build time max 3mins locally; build pipeline max 5mins.

1.2. Service Types

- single codebase, or multiple (classpath isolated) codebases
- single service (pseudo-monolith) vs multiple services (isolation, performance)
- TODO read and summarize: https://nordicapis.com/whats-the-difference-microservice-macroservice-monolith/

1.2.1. Monolith

- one codebase, one deployable
- GOOD: low complexity (versioning, service location) and high speed (calls via code in same process instead network)
- BAD: change-resistant, slow, …

1.2.2. Modulith

- one codebase, basically just a monolith but a bit cleaner packaging; domain wise, not tech
- all the disadvantages of a monolith
- degrees:
- low version: only java packages (enforce package import; use spring modulith)
- mid version: maven sub-modules (sub-projects in gradle) or external repos with dependency on them
- high version: classpath isolated libraries (OSGi bundles, plugin architecture; extreme low coupling)
- this seems to be the only one counteracting the monolith's nature of low/mid version

1.2.3. Macroservices

- bigger sized microservices
- a bit less (manageable) complexity (pros of monolith) and still good enough to gain from the benefits (pros of modulith)

1.2.4. Microservices

- one very small codebase to one very small deployable
- GOOD:
- one service down, rest of application still runs
- very quick build time; code very isolated, small, thus easy to change
- BAD:

- complexity, service orchestration, versioning

1.2.5. Nanoservices

- deploy functions (AWS lambda)
- seems too extreme; complete different approach
- https://medium.com/@shrinit.poojary12/microservices-vs-nano-services-a-detailed-comparison-993453e65b8c

1.3. Layering

How the application can be **split and grouped** with different aspects in mind, each with their own pros and cons. What kind of logical layering (represented by a consistent naming pattern) and technical subdivisioning is used. Whether a subdivision is implemented as a submodule (same VCS repo), or an external dependency (own VCS repo).

1.3.1. Packaging

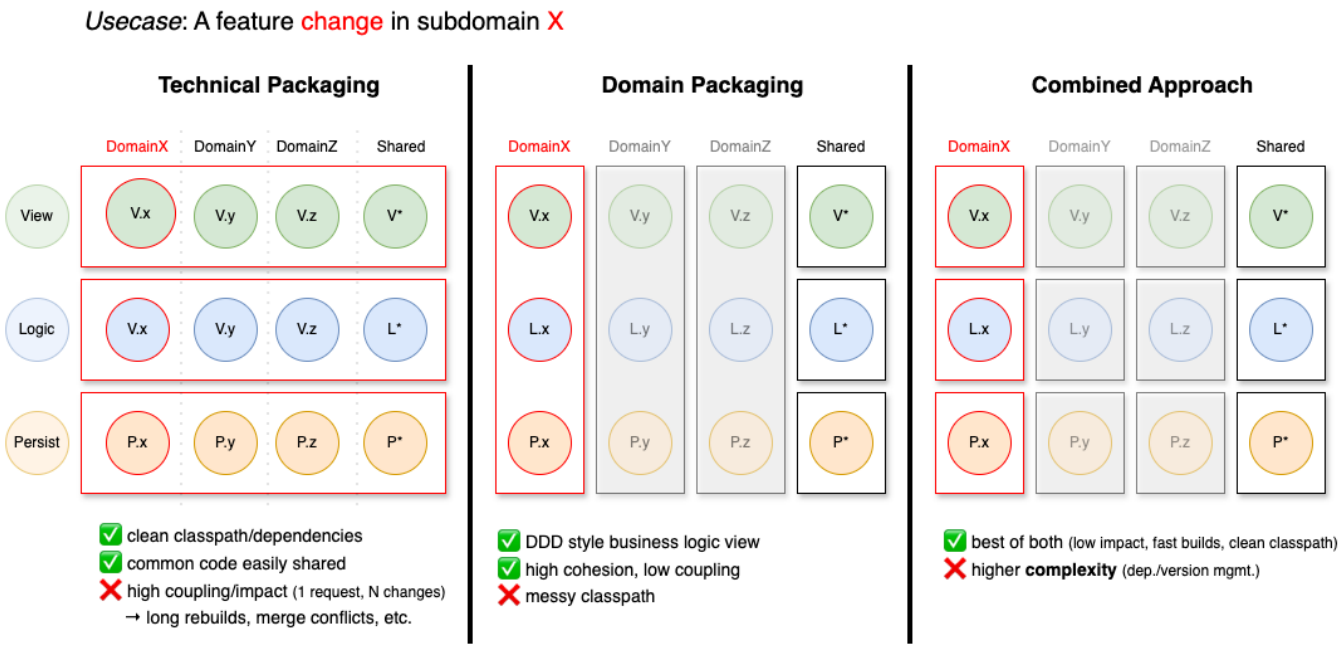

Figure 3. The degree of impact on the application depending on the choice of packaging strategy

Given the usecase of having to add a new field to a specific domain. All three layers need to be touched: view, logic, and persistence for that domain.

Three approaches are possible:

1. Technical Packaging
    - This is what we usually do; down to the extreme of packages like **`exception`** or even **`enum`**.
    - In terms of single responsibility this is doubtful (simply looking at the import statements).
    - When touching one aspect, all layers (with all domains), basically the whole application was changed (no caching/reusing of already built modules).
2. Domain Packaging
    - Java's package/namespace idea of reversed DNS names was initially intended for domain packaging.
    - E.g. When a feature login needs to be implemented, only classes within the domain

package **`auth`** will be touched (view, logic, and persistence in one package).
    - But now the whole classpath is polluted with all kind of different technical aspects (view controller next to DB repositories)…
3. Combined Approach
    - To split it up and we gained the best of both worlds; with some added complexity.

Final thought:

- This principle not only applies to packages, but also to services; in the big and in the small.
- There is no absolute right way of doing it; any solution will have pros and cons.
- Often the vertical slice is represented as a microservice, thus within it the domain is already split (it could be split even further though; too much?), thus only tech packaging relevant.
    - Tech split within domain split.
- And always then the need (if no horizontal layer) to create a shared library for each layer…

1.3.2. Subprojects

Each architectural layer is represented by a Gradle sub-project (or sub-module as called in Maven terminology).

Main Subprojects:

- app
- view
    - view-routing and view-controller-[api/impl] could be merged into one…
- logic-api
- logic-impl
- extern
    - extern-api
    - extern-impl (HTTP, SFTP)
    - extern-stub
    - extern-model (?) - generated source code (YAML, WSDL)
- persistence
    - persistence-api
    - persistence-impl (is actually persistence-exposed)
    - once there was a persistence-stub, but it turned out to be useless (already in full control)

Other Subprojects:

- client-sdk (kotlin MP)
    - sdk
    - models
    - tests?
- openapi-gen (resides in shared for now, needs to be externalized)

Test Subprojects:

- itest
- etest: separate GIT repo (maybe dep on client-model)

Shared Subprojects:

- test
- logging

1.3.3. Client-SDKs

What is it?

- A (provided) client library to integrate a service (backend, "extern")
- It contains:
  - DTO models; serializable data transfer object.
  - API Client interfaces; either ready-to-use controllers or simply header interfaces.
  - Reusable wiremock preparations (contract alignment)
  - Reusable test instances for property based testing.

Why?

- It reduces integration-effort (code to write and maintain).
- It takes risk of writing integration tests based on wrong assumptions (contract).
  - Capturing behavior in a formal way which can be tested against.

Code generation:

- Preferrably most is generated by an OpenAPI generator (contract enforcement).
  - Not everything is doable with code generation (at least not with realistic effort); some needs to be handwritten.
- It is preferrably available in all used client languages.
  - Different client technologies exist; in different versions.
- Alternative: Provide custom generators to be used.

Best Practices:

- Every service should provide such a SDK; if not, implement ourselves.
  - The code resides in its own repository; separately built and versioned.
- We maintain a client-useful (confirmed benefit) SDK for our API ourselves.
  - We potentiialy used it ourselves in tests, http-controllers, etc.

1.3.3.1. Layer Granularity

Status: Draft
Created: 5.12.2025
Author: Seepick

Should the view-routing and view-controller-* modules be separate or merged?

- if testing strategy is testing from http (ktor test engine) then it can be merged.
  - if testing from controller (no ktor involved, plain code, simpler but less realistic/higher risk, require complementary tests and integration of those layers) then it should be separate.
- current situation feels like adding a costly feature without using its benefit
  - the modules are separated and the itests are not using the controllers (but HTTP ktor test engine)

Separate:

- very clean
  - view-route only purpose interact with web framework and leave
  - easy to test routes
- adds complexity (already heavy due to route - controller - domain - persist/extern)
  - controller interface, controller impl, register koin, …
  - but enforces clean architecture!
- itests could use view-controllers instead using HTTP (from cucumber looks the same, just faster/easier setup)

Merged:

- faster development; less code/complexity
- unit (integration, as they use ktor) test for routes will be heavier.

1.4. Technology Choices

JVM based, not .NET based, because "Java is dead, long live Java". The (Java) ecosystem is fantastic, opensource, mature, and stable. The (Java) programming language is old and outdated. Other languages are nowadays running on the JVM, like Kotlin and Scala. Especially Kotlin is fully interoperable with Java and thus all the tools can be reused and for developers a smooth migration path exists.

ADR - Language Choice

Status: Draft
Created: 5.12.2025
Author: Seepick

Which programming language to choose?

- Java or Kotlin?
  - not really any other: Definitely no Scala, Clojure, … and no non-JVM languages (Go, Rust, Python, C#, …).
- interoperability; always possible to fall back to java code and "java native libs"
- using Lombok is an indication we want Kotlin:
  - from plain java, to Lombok, to kotlin
  - native data classes support, properties, prop-initializing-ctor; from final to val

Main points for Kotlin:

- Similar to java; fully interoperable (could use Spring)
- concise (less boilerplate, less verbose); cleaner/less code, faster development
- Null safety built into the type system
- Extension functions
- Functional (functions as first-class citizens; higher-order functions)
- Immutability by default (collections)
- JetBrains supported, tooling (IDE!), ecosystem

Side points for Kotlin:

- Concise lambda definition (custom DSLs)
- Type inference, smart casts
- Bypass type erasure with inline functions and reified generics (have type parameter available at runtime, no more typeOf magic)
- Default and named parameters
- String templating
- Coroutines (async)

Some things Java was able to catch up a bit (yet sometimes a bit poorly implemented): lambdas, method reference, pattern matching, records

Downsides for Kotlin:

- Not as popular (tools, libs, documentation, experience/skill)
- Compiler is not as highly optimized as Java's
- 100% interoperable regarding the language itself, but with some (older) frameworks there are some incompatibilities in paradigm

ADR - Webframework Choice

Status: Draft
Created: 5.12.2025
Author: Seepick

We need a web framework to communicate JSON-over-HTTP.

- The web framework is usually also dictating a bigger part of the application: dependency injection, configuration, etc.
    - Especially with heavy-weight frameworks like Spring Boot and the enterprise implementations are covering more than just "being a fancy servlet".

Options:

1. Spring Boot/Web
    - Most common in JVM world.
    - very opinionated (convention over configuration like)
    - Very much like maven, “super strict, either our or no way”
    - Its pro is its con: beneficial at beginning, but no power user custom things (fighting the convention)
    - Difficult to start only parts of the application (testing), in isolation (parallel testing; @DirtiesContext), due to classpath scanning and annotations usage.
2. Ktor
    - Lightweight, modern (coroutines), multiplatform (typescript), server and client
    - Less popular (getting better), less mature, smaller community (documentation).
    - Much needs handwritten; some good (code, control), some bad (openapi generation, jpa orm)
    - Everything is code (instead of annotations), there is a beautifully simple way to do it, and so extremely flexible, that every test strategy we can imagine can be implemented.
3. JEE JAX-RS
    - Jersey, ReST-easy
4. Quarkus
    - More popular, more lightweight than JEE implementations.

Spring and Kotlin?

- more and more attention from Spring towards Kotlin; more support
- see kofu for spring experimental support to make it look like ktor
- also interesting: Spring Kotlin DSL

Conclusion:

- We use Ktor

Persistence Technology

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- We need to choose a persistence technology for our application.
- A clear distinction between the needs for production vs test/development (local startup).
- We choose for a relational database…
    - No NoSQL store (MongoDB) or in-memory cache

Requirements:

Production DB:

- Available in corporate environment.
- If performance (searching/filtering) is an issue, maybe an elasticsearch index could be useful?

Test/Dev DB:

- Lightweight (in-memory)
- Fast startup and execution time
- No setup costs (installing, configuration, etc)

Options:

Production:

1. Oracle
2. Postgres/MySQL
3. MS SQL Server

Test/Dev:

1. H2
2. HSQLDB
3. Sqlite

Chapter 2. API Design

- OpenAPI even needed?
  - The spec is the source of truth, the holy contract
  - Especially multiple consumers (always at least 2: 1x FE and testers)
  - OpenAPI generators can keep contract in sync with code
- Show SwaggerUI as endpoint (display both UI and yaml-spec); consumer convenience

2.1. Filter ReST Resources

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- the user (via the frontend) needs to be able to filter bigger datasetes
- different fields have different types require different operators
  - general: equal, not equal
  - numeric: greater/lesser (or equal); between is done by adding both ;)
  - enum: (with)in
- operators are all in conjunctive form (coupled by AND)

Proposal:

Simple:

- `/crystals?filter=author eq 'Peter'`
- `/crystals?author=eq:Peter`
- `/crystals?weight=gt:3&weight=lte:10`
- `/crystals?state=in:FOO,BAR`

Complex:

- `/crystals?filter[0].property=owner.name&filter[0].operator=equals&filter[0].value=Peter`

Radical other approach would be to turn it into a POST (PUT?) request and provide a request body (full control). It feels like we are trying to squeeze a proper payload into query parameters (feels wrong, a smell we are misusing something).

Links:

- https://restfulapi.net/api-pagination-sorting-filtering/

2.2. Generate Custom OpenAPI Code

Status: Draft
Created: 1.12.2025
Author: Seepick

Reason:

- We need to keep the API spec (YAML) and the code (Kotlin) in sync; we don't want to rely on doing this manually.
- Thus, establish an automatism via generation.
- We need something that fits our needs: modern, up2date, high quality code and libs.
- Current (Kotlin) generators are out of date (unusable), thus a custom generator is required.

Alternatives:

- Write some tests to verify sync of spec and code.
- No, as we want immediate feedback (compile time) about correct implementation.
- Use Java generators, they are more modern and Kotlin-Java interoperability works well.
- It would not clash as Java generated clients use different libraries (even when outdated).

Implementation requirements:

- Kotlin data classes with Kotlinx serialization annotations.
- Ability to hook specific type serializer (provided or custom).
- Think of **`java.time.LocalDateTime`** and enums.
- Generated separately (**`view-model`** subproject) from server routes; a unique requirement not provided by default generators.
- Interfaces for Ktor server.
- Some route providing interface types could be injected into Koin.
- Only bare skeleton mandatory (HTTP method and path), everything else optional (could go crazy with details).
- Client generation is optional.

Resources:

- https://www.baeldung.com/java-openapi-custom-generator
- https://deepwiki.com/OpenAPITools/openapi-generator/5.1-creating-custom-generators

2.3. Paginate ReST Resources

Status: Draft
Created: 1.12.2025
Author: Seepick

Approaches:

Offset:

- offset, limit; skip, take
- based upon it can implement page-based pagination (could support both actually)
- bad for big datasets; DB has to scan through rows

Others:

- Keyset pagination, seek method
- for example via **`since_id`** and a limit/take param
- only for IDs with auto-increment, or timestamps
- Time-based pagination
- for analytics/log monitoring only
- Cursor-based pagination
- like a bookmark, a backend-determined value not necessarily linked to any data fields (contrary to keyset)
- more complex to implement
- also return the "next cursor" in the response

Best Practices:

- clearly document pagination mechanism (openAPI, provide examples)
- don't implement pagination for: A) small datasets and B) fast changing data
- use standard names
- provide page metadata:
- total pages, current page
- total items, items in page, size (request page size)

- has more
- provide navigation _links (HATEOES), next/previous/first/last
- either in metadata payload
- or in header: Link: <http://localhost:8080/api/books/paged?page=1&size=10>; rel="self"
- reuse test logic: each paginated endpoint needs to have the same set of tests

Questions:

- either use or at least get "inspired" by: https://github.com/perracodex/exposed-pagination
- how strict or lenient to be?
- negative numbers (skip -1, take 0)?
- take more than existing (page > totalPages)
- restrict a maximum take/limit param?

Resources:

- TODO get inspired by: https://github.com/perracodex/exposed-pagination
- https://www.merge.dev/blog/rest-api-pagination
- https://www.merge.dev/blog/api-pagination-best-practices
- https://restfulapi.net/api-pagination-sorting-filtering/

2.4. Secure HTTP Endpoints

Status: Draft
Created: 1.12.2025
Author: Seepick

How is authentification and authorization handled?

- IAM/LDAP
- Central security auth gateway in front
  - A reverse proxy (nginx) for auth (more complex pipeline)
  - Services only redceive an already verified user ID header (auth-agnostic)
  - Simplifies lots of code
  - Faster build time ("IAM policy seeding" can take a lot of time)
  - Easier test setup (happier testers)
- Separation of concerns: make the "children slimmer/lighter"
- Endpoints are configured centrally (?); the obligatory "auth matrix" ;)
- OAuth2 vs JWT?

2.5. Sort ReST Resources

Status: Draft
Created: 1.12.2025
Author: Seepick

About:

- call it sort or orderBy
- **`orderBy=author,title`**
- sort direction: asc/desc suffix
- **`orderBy=author asc,title desc`**
- or plus/minus: **`orderBy=+author,-title`**
- default to asc if not defined

Error Handling:

E.g.:

Example from a fictional API

```
{
  "error": {
    "code": "InvalidOrderByExpression",
    "message": "The property 'author' in the orderby expression is not sortable",
    "details": [
      {
        "code": "UnsupportedSortProperty",
        "target": "author",
        "message": "Sorting by 'author' is not supported. Supported sort properties
are: ['id', 'title']"
      }
    ],
    "target": "orderby",
    "internal": {
      "trace-id": "00-d24f899d9c8a5a428fddd39399e7f58e-5c8a8949fcdd9a42-01",
      "timestamp": "2024-02-20T15:30:45.123Z",
      "request-id": "123e4567-e89b-12d3-a456-426614174000"
    }
  }
}
```

Links:

- https://restfulapi.net/api-pagination-sorting-filtering/

2.6. Versioning of HTTP Endpoints

Status: Draft
Created: 1.12.2025
Author: Seepick

Options:

1. using header: **`Accept: application/vnd.diamond.crystal.v1+json`** and **`Content-Type: …`**
    - Keeps the URLs clean; metadata in headers
    - But not so obvious at first, requires additional logic to be implemented
2. encode in path: **`GET /api/v1/crystals`**
    - out-of-the-box supported by caching

Also:

- deprecate (TODO: tag OpenAPI spec as such, generate warning in code possible?!) and communicate
    - Keep track (logging) of usage to know when to turn off

Links:

- https://dev.to/abhivyaktii/api-versioning-using-headers-a-clean-and-flexible-approach-218p

Chapter 3. Application

3.1. Techstack

Library choice guidelines:

- Kotlin-idiomatic (DSLs, functional)
- Asynchronous (native coroutine support)
- Alive project (recent commit to repo)
- Non-intrusive (clean code, no coupling)
- Code over Config (incl. annotations)

As a golden rule, there should never be any **non-production dependencies** (test, stubs, localdev, etc like H2) in production code. Instead activate profiles which customize the build (dependency tree). E.g. for tests (presistence-tests and itests) different wiring

3.1.1. Libraries

A list of all used dependencies can be found at: buildSrc/Deps.kt

```
object Deps {
    val arrowCore = "io.arrow-kt:arrow-core:${Versions.arrow}"
    val jsch = "com.github.mwiede:jsch:${Versions.jsch}"
    object database {
        val h2 = "com.h2database:h2:${Versions.h2}"
        object exposed {
            private fun make(artifact: String) = "org.jetbrains.exposed:exposed-
$artifact:${Versions.exposed}"
            val jdbc = make("jdbc")
            val datetime = make("java-time")
        }
    }
    // ...
}
// Reference your dependencies as such:
dependencies {
    implementation(Deps.database.exposed.jdbc)
}
```

3.1.1.1. General

Gradle
: Build system to run tasks such as compiling, testing, and packaging ("*newer Maven*", using its sophisticated dependency management).

Kotlin
: General purpose programming language running on the JVM ("*newer Java*", fully interoperable).

SonarQube
: Quality gate for code analysis, test coverage and more.

3.1.1.2. Core Libs

Ktor
: Lightweight web framework ("*newer Spring Web*").

Koin
: Dependency injection, actually service locator, framework ("*newer Spring Framework*", neither classpath scanning nor annotations use).

Kotest
: Modern flexible testing framework integrating with the JUnit engine ("*newer JUnit*")/

Exposed
: Typesafe SQL library (not a full fledged ORM like JPA/Hbernate).

Liquibase
: Database migration library using XML.

Arrow
: Functional library used for better exception handling via `Either`.

Hoplite
: Typesafe loading of application configuration.

3.1.1.3. Support

Detekt
: Static code analysis together ("*newer CheckStyle*")

Ktlint
: Auto-formatter, integrated via Detekt.

Kover
: Test coverage tool, generating JaCoCo compatible XML reports.

3.1.2. Library Choices

3.1.2.1. Database Access Library

Status: Accepted
Created: 1.12.2025
Author: Seepick

Context:

- We need a way to access relational database from Kotlin code.
  - We talk about DB access (as in JDBC), not a full ORM solution.
- Question is, which library to use?

Requirements:

- Needs to be type-safe (no "stringly-typed" expressions)
- Little/no code generation magic
- Kotlin-idiomatic (coroutines, DSLs)
  - For using coroutines we would need a non-blocking implementation (no JDBC)

Options:

1. Exposed by JetBrains
   - Seems production ready; mature and stable enough
   - Lightweight and typesafe
2. KTorm
   - ORM mapper for Kotlin, pure JDBC
3. JPA / Hibernate
   - Heavyweight, not Kotlin-idiomatic
   - Lots of magic, code generation, proxies, etc.

Choice:

- Exposed
- It fits with the rest of the used ecosystem.
- We use the conventional DSL, not the DAO API (seem unfinished).

Example of Exposed's (unfavored) DAO API

```
// definition
class CrystalDaoEntity(id: EntityID<UUID>) : UUIDEntity(id) {
    var weightInGrams by CrystalTable.weightInGrams
    companion object : UUIDEntityClass<CrystalDaoEntity>(CrystalTable)
}
// usage
CrystalDaoEntity.findById(id)
CrystalDaoEntity.new(UUID.randomUUID()) {
    weightInGrams = 42
}
CrystalDaoEntity.findByIdAndUpdate(id) {
    it.weightInGrams = 42
}
```

3.1.2.2. Scheduler Library

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- we need to run jobs regularly in a cronjob way
- no spring scheduler available

Options:

- Quartz https://www.quartz-scheduler.org
- enterprise (heavy, slow) scheduler; the "platzhirsch", used to be the one and only; decades
- but the pros comes with cons: outdated, verbose APIs
- features: persisted in DB (requires 10+! tables), retry, clustering, prioritization etc
- no monitoring (grafana, jaeger)
- JobRunr https://www.jobrunr.io
- lightweight, modern; RDBMS (and NoSQL)
- features: distributed, dashboard, retry
- db-scheduler https://github.com/kagkarlsson/db-scheduler
- lightweight: performant, minimal deps; still offer what's need: persistence, cluster

maybe there is a mature enough kotlin library which leverages coroutines?

- KtScheduler https://github.com/Pool-Of-Tears/KtScheduler
- seems like a small hobby project…

Links:

- https://www.jobrunr.io/en/blog/2024-10-31-task-schedulers-java-modern-alternatives-to-quartz/

3.1.2.3. Access SFTP service

Status: Draft
Created: 1.12.2025

Author: Seepick

Context:

- JSch is very old and very outdated
- Use forked and updated **`com.github.mwiede:jsch`** instead of original **`com.jcraft:jsch`**

3.1.2.4. DateTime Library

Status: Draft
Created: 1.12.2025
Author: Seepick

Use java-time or kotlin-time? (joda not needed anymore)

Java:

- GOOD: robust, well-known, well-integrated with libs (serialization, DB)
- BAD: cumberesome
- BAD: not supported by kotlin-idiomatic libs

Kotlin:

- GOOD: lightweight, probably/most-likely kotlin idiomatic; modern API
- GOOD: integration with kotlinx-serialization and exposed
- BAD: not all (java) libs support it
- BAD: higher barrier of entry for others
- any benefit from its multiplatform nature?

Conclusion:

- unstable API is too dangerous for production!
- NoSuchMethod error, incompatible versions on classpath
- kotlin is not well integrated yet (kotest)

3.1.2.5. DTO Mapping Library

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- We want a clean representation, specific to a layer, of our data model.
  - Consequence: Many different representations of the same (similar) thing.
  - Consequence: Lots of (tedious) conversation (mapping/transformation) necessary.
- Solution: Library which does that for us.

Options:

- Mapstruct
  - Using annotations for code generation
  - Generated code outperforms hand-written/reflection-based mapping solutions
  - Typesafe checks; customization; more the heavy-weight
- ModelMapper
  - Using reflection (slower, less typesafe)
  - Tries as much as possible by default (same name/type)
  - Easy and simple (for basic mappings), convenient, flexible; light-weight

Resources:

- https://www.javacodegeeks.com/2025/01/mapstruct-vs-modelmapper-a-comparative-

analysis.html

3.1.2.6. Serialization Library

Status: Draft
Created: 7.12.2025
Author: Seepick

A marshaller/serializer is needed for JSON support.

Options:

1. Jackson
    - Very mature and well-known
    - Some "hickups" with Kotlin's different nature
2. gson
3. Kotlinx Serialization
    - Kotlin-idiomatic; reflection/annotation based
    - Simple to write custom de/serializers
    - Multiplatform support (client SDK for frontend)

Aspects:

- Handling of Java's datetime
- Support of value classes (typesafety + inlining)

Conclusion:

- Kotlinx Serialization is best fitting

3.2. Startup Arguments

- **`java -jar diamond.jar printConfigOnly`** - will print the read configuration object for verification
    - Used in **`./bin/test_apponfig.sh`** to verify output

Sample toString representation of the application configuration

```
EnvConfig(
  ktor=KtorConfig(port=12),
  database=DatabaseConfig(
    jdbcUrl=db_url,
    username=db_user,
    password=****
  ),
  extern=ExternConfig(
    postsServiceBaseUrl=postsUrl,
    sftp=SftpConfig(
      remoteHost=sftpHost,
      port=22,
      username=sftpUser,
      authIsPassword=true,
      authPasswordOrPrivateKeyPath=****,
      knownHostsFilePath=knownHosts,
      strictHostChecking=true
    )
  )
)
```

3.3. Application Configuration

Status: Draft
Created: 1.12.2025
Author: Seepick

- config as ENV vars or centralized config manager?

Links:

- https://softwarepatternslexicon.com/kotlin/microservices-design-patterns/configuration-management/

3.4. Environment Variables

The following lists all configuration values passed to the application as environment variables. It was automatically generated by **`nl.uwv.smz.diamond.app.ConfigDocWriterApp`** using reflection to extract information.

▼ Environment Variables Table:

Param	Type	Default	Description
`DATABASE_JDBCURL`	string	-	JDBC driver URL
`DATABASE_PASSWORD`	string	-	DB password
`DATABASE_USERNAME`	string	-	DB username
`EXTERN_POSTSSERVICEBASEURL`	string	-	Base URL for the external posts service
`EXTERN_SFTP_AUTHISPASSWORD`	boolean	-	Whether using password or private key.
`EXTERN_SFTP_AUTHPASSWORDORPRIVATEKEYPATH`	string	-	Either password or private key.
`EXTERN_SFTP_KNOWNHOSTSFILEPATH`	string	-	Path to SSH known hosts file.
`EXTERN_SFTP_PORT`	integer	22	Port of the SFTP server.
`EXTERN_SFTP_REMOTEHOST`	string	-	Host of the SFTP server.
`EXTERN_SFTP_STRICTHOSTCHECKING`	boolean	true	Disable security check; do NOT activate in production; pretty please.
`EXTERN_SFTP_USERNAME`	string	-	Login username.

Param	Type	Default	Description
KTOR_PORT	integer	8080	Webserver HTTP port to listen to

3.5. Gradle

- Guiding Principles:
  - Keep build log output silent; don't pollute it; don't be verbose (by default)
  - Keep all warnings down to zero immediately (0-warning policy applies here as well).
- Sharing build logic via the **/buildSrc/** directory.
  - See: https://docs.gradle.org/current/userguide/sharing_build_logic_between_subprojects.html

3.5.1. Gradle Properties

- Enable certain flags (enabled via gradle properties and on CI), it will enable more tests/reports (which take longer and require more local setup)
- Pass them either as system variables **-D** (as gradle properties sometimes doesn't work **-P**).

As seen in buildSrc/GradleProperty.kt:

- **diamond_version** - application version string
- **diamond_branch** - Git branch name
- **isCi** - flag to indicate running in a build pipeline (false by default, assuming local builds)
- **enableOwasp** - Enable additional security report (takes some time, thus disabled by default)
- **diamond_version** - application version, defaults to **0** if not specified
  - displayed in info endpoint, used in generated documentation
- **runTestcontainersTests** - see task below
- **runEtests** - see task below

3.5.2. Gradle Tasks

- Build an executable JAR:
  - **./gradlew :app:buildFatJar -Pdiamond_version="42"**
- Check for outdated dependencies:
- **./gradlew dependencyUpdates**
- Deploy container image to local Docker registry.
- **./gradlew publishImageToLocalRegistry**

Test:

- Run tests using testcontainers (requires Docker daemon to be running)
  - **./gradlew test -PrunTestcontainersTests=true**
- Run Karate end-to-end tests.
  - **./gradlew test -PrunEtests=true**

Documentation:

- Generate documentation for available ENV properties:
  - **./gradlew :app:generateConfigDoc**

- Generate software documentation as PDF:
  - `./gradlew :doc:SoftwareDocument:asciidoctorPdf`
- Generate software documentation as HTML:
- `./gradlew :doc:SoftwareDocument:asciidoctor`

3.5.3. Manage Dependencies

Status: Draft
Created: 1.12.2025
Author: Seepick

Which way to properly manage gradle dependencies?

Problem description:

- versions repeatedly being declared, danger of mismatch (same library, different versions)
  - especially true for multi-module projects
  - leading to runtime error due to binary incompatibility (NoSuchMethodError, etc.)
- cumbersome as in lots to type and repeat (we want to be concise and fast)

Requirements:

- not abstracting/hiding too much (e.g. one god dependency), but be explicit/transparent enough for understanding
  - be concise enough, while capturing the essence for clarity (intuitive comprehension) sake
- not too complex, no over-engineering, just get it "good enough"
- deal with a huge amount of versions/dependencies/plugins
- custom plugin declaration (to share common code as extensions/compositions) can use it as well

Alternatives:

Either: 1) buildSrc custom code or 2) use gradle's standard version catalog.

Custom `buildSrc` Code:

Pros:

- it's plain kotlin code with all its advantages
  - refactoring safe
  - immediate feedback from compiler if something is wrong
  - go to source with CTRL+click (javadoc)
- full control and flexibility (custom made solution)
  - e.g. group versions and dependencies
- already used for custom plugins
  - BUT: it is NOT possible to use things from the catalog for those

Cons:

- it's custom made for something a default solution already exists
- doesn't support view usages

3.5.3.1. Gradle Version Catalog

- https://docs.gradle.org/current/userguide/version_catalogs.html

Pros:

- it's a out-of-the-box supported solution, people know it, well documented
- allows for grouped dependencies, so-called libraries

Cons:

- TOML syntax is not as useful as code (auto-completion, type feedback, etc.)
- going into source implementation/definition not possible (CTRL+click), cumbersome
- if changing/refactoring, then step-by-step errors by each build, cumbersome

Example of a Gradle version catalog

```
[bundles]
ktor = ["ktor-core", "ktor-json", "ktor-foobar"]

[plugins]
short-notation = "some.plugin.id:1.4"
versions = { id = "com.github.ben-manes.versions", version.ref = "manes-versions" }
```

Conclusion:

- both support grouping and a nested-hierarchy to control a vast amount of declarations
- both support auto-completion
- custom plugin declaration is fucked
    - neither the custom code approach, nor the version catalog supports it (too early in build phase i assume)

3.5.4. Gradle Kotlin DSL

Status: Draft
Created: 1.12.2025
Author: Seepick

Context: * obviously not groovy (cumbersome) * as project is already in kotlin, makes sense to configure gradle also with kotlin * has become stable enough the last years (support, plugins/configuration, sufficient documentation on the internet)

3.6. Documentation

3.6.1. Documentation Tool

Status: Draft
Created: 1.12.2025
Author: Seepick

Options:

- Markdown
    - Well known and well supported
    - Might not provided sufficient functionality for a more complex/bigger documentation project
- Asciidoc
    - Somewhere between Markdown and LaTeX in usability/support and functionality/complexity
    - GitHub can also render it, just like markdown, yay
- LaTeX
    - Great for writing books, but definitely poses a too high barrier of entry for people unfamiliar with it
    - Local latex installation is far more than needed compared to provided infrastructure for other technologies

3.6.2. Diagram Drawing Tool

Status: Draft
Created: 1.12.2025
Author: Seepick

Requirements:

- programmatic (source code, version trackable)
- but high barrier entry
- no WYSIWYG, thus not suitable for complex (and "pretty") diagrams
- defined along adoc
- easy access (no visio)
- user/rights/auth
- web based

Options:

- plantuml
- mermaid
- ...?

PlantUml:

Install `dot` binary from https://graphviz.org/download/ to be able to build diagrams with code.

```
[plantuml,inlineUml,svg]
....
@startuml
[Inline] --> Interface1
[Inline] -> Interface2
@enduml
....

Or included:

[plantuml,includedUml,svg]
....
include::includes/diagram.puml[]
....
```

- With markdown (GitHub) also PlantUML possible "somehow", but not really...
  - https://gist.github.com/noamtamim/f11982b28602bd7e604c233fbe9d910f

Chapter 4. Coding

Kotlin.

- list the bad things as well; honesty, expection mgmt, credibility.
- intellij support; crash; young

4.1. Coding Philosophy

- fail early, fail fast
- e.g. if application config something is configured wrong/missing
- validate data (front incoming HTTP, back external systems/DB, immediately throw)
- code first / code over config
    - no strings/annotations, no declarative xml/json/yaml/properties
    - just code, be in control, flexible, safe (compiler reference check)
- no classpath scanning, reflection, other look-ups/service-locator, no auto-magically something
- avoid intrusive frameworks (the systems serves us, we don't serve the system)
- functional programming
- pure functions: stateless, side-effect free
- code as named expressions
- prefer single-expression-method `fun foo() = doSome().doOther().also { done(it) }`
- absolutely no local files (IDE specific, build generated, …)
    - not checked in, even existing put on .gitignore
    - all auto-configured by build system/gradle
    - don't interfere with each other; give devs freedom - no coupling
- single responsibility (of package, class, method/function)
- clear names, self-explanatory, capturing its essence (single responsibility)
    - no abbreviations, no short cryptic names

4.1.1. Best Practices

- Strive for functional, e.g.: **pure functions**:
    - return values are identical for identical arguments (no variation with static variables, non-local variables, mutable reference arguments or input streams, i.e., referential transparency)
    - the function has no side effects (no mutation of non-local variables, mutable reference arguments or input/output streams)
- no overloaded methods, provide different name
- instead of regular methods → "named expressions"
    - functions as named codeblocks, lead to shorter (easier to comprehend) methods
    - always use "=" after method signature; single expression body
    - no state, no "val" keyword, all just immediate execution (more functional)
    - forces to write very short methods, easy to read and comprehend
    - methods have names, they create logical structure of arbitrary code

4.1.1.1. Conciseness

Using type inference and single expression functions, function declarations can be shortened

and made more readable:

Ommitting the return type

```
fun `BAD explicit`(): String = "foo"

fun `GOOD inferred`() = "foo"
```

Using lambda wrappers

```
fun `BAD increase indentation`() {
    transactional {
        repo.update()
    }
}

fun `GOOD single expression`() = transactional { ❶
    repo.update()
}
```

❶ Sometimes needed to override inferred type to `: Unit`.

Using standard functionals

```
fun Service.`BAD return`(): Service {
    setFoo("bar")
    return this
}

fun Service.`GOOD apply`() = apply {
    setFoo("bar")
}
```

4.2. Code Samples

Source of **`healthRouting.kt`** from the **`view-routing`** module

```
package nl.uwv.smz.diamond.view.routing

import io.ktor.server.application.Application
import io.ktor.server.response.respond
import io.ktor.server.routing.get
import io.ktor.server.routing.routing
import nl.uwv.smz.diamond.view.controllerApi.HealthController
import org.koin.ktor.ext.inject

internal fun Application.installHealthRouting() {
    val controller by inject<HealthController>()
    routing {
        get("/health") {
            call.respond(controller.fetchHealthReport())
        }
    }
}
```

4.3. Logging

4.3.1. Logging Configuration

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- following principle: code > config
    - more flexibility, more control, more sophisticated, more robust
- programmatic logging configuration
- dev/test different from default-prod
- provide mechanism to override(?), maybe even
- TODO: runtime, to "debug" = increase log; in running prod

4.3.2. Logging with Kotlin

Status: Draft
Created: 1.12.2025
Author: Seepick

Logging evolution:

Initial approach with performance in mind

```
if (log.isDebugEnabled()) {
    log.debug("some message " + message + " was returned")
}
```

This way we only create the string if it is really necessary, bad. But it's very verbose, to do it correct and concise; bad.

Then we switched to a **`String.format`**-like approach to fix this, to always be able to invoke the log method without performance impact:

Less verbose, but still performant

```
log.debug("some message %s was returned", message)
```

This two issues (conciseness, performance) but still was not readable, especially when logging longer messages with multiple values. The displacement-nature of these kind of formatting approaches is just not doing it…

Finally we are doing it nicely, using Kotlin's string templates (interpolation) which are basically string concatenations, and the ease of lambda declarations (deferred/lazy evaluation):

Readable and performant

```
log.debug { "some message $dto was returned" }
```

Logger Declaration:*

Thanks to **`io.github.oshai:kotlin-logging-jvm`** (former "mu-kotlin") the declaration of a logger is as simple as it can be:

Implicit log factory creation

```
private val log = logger {}
```

It will look up the surrounding class and thus use the right logger factory itself. It uses Slf4j underneath.

The actual logging implementation is done by Logback (the better Log4j).

4.4. Local Development

The goal is to only have to check out the sourcecode, and run a single command to start the application up in a local model. No further setup must be needed (no docker, no special files created). There is no required readme to be read, no manual or instructions; it just works out of the box.

- run **`nl.uwv.smz.diamond.app.LocalDiamondApp`**
- Postman collection can be found at: **`/config/Diamond.postman_collection.json`**
- There are a lot of useful (required) build scripts in **`/bin/*.sh`**

4.4.1. IntelliJ Plugins

The following plugins are recommended to have a smooth development experience:

- Gradle
- EditorConfig - autoconfigures styling
- detekt - configure with **`/config/detekt.yml`**
- ktlint - auto-formatter
- kotest - test framework integration
- Markdown - some internal documentation files use it
- AsciiDoc - documentation written in it
- SonarQube for IDE - connector to quality gates
    - Configure with SonarQube Cloud project: https://sonarcloud.io/project/overview?id=seepick_diamond
- Gherkin - Test specifications support
- Cucumber for Kotlin - Step definitions; regexp
- Cucumber for Java - not sure, i guess it adds to it (?)
- Cucumber+ - Better support
- Cucumber Table Mapping - Specific support
- kotlin-fill-class - Useful inline code generator

Other:

- plantuml4idea (graphivz/dot required)

4.5. Code Generation

4.5.1. Generating Sources

Status: Draft
Created: 6.12.2025
Author: Seepick

Context:

- We use specifications to document the contract, provided by ourselves and third parties.
- Primarily OpenAPI YAML and WSDL XML files.

Should we regenerate sources each time and host the code next to our own, or should be pre-generate and store them in a dedicated repository?

Options:

Re-generate:

- GOOD: Everything is self-contained; simple, easy.
- BAD: Slows down the build process due to regeneration despite no API changes (caching?).
- BAD: Exclusion configuration necessary for static code analysis, coverage, etc. (different rules apply to them)

Pre-generate:

- GOOD: Faster build times.
- GOOD: A change of the API contract must be a conscious act
    - Leads to more safety/stability
    - But it's also a BAD argument: slower development time, more complex!
- For 3rd party APIs, we would expect a given SDK anyways… so we do it ourselves.

Decision:

- Pre-generate!
- Divide and conquer: Manage the bigger complexity by outsourcing.
- Split things apart, separation of concerns.

4.5.2. Generate Persistence Code

Status: Draft
Created: 6.12.2025
Author: Seepick

Context:

- liquibase migration strategy…
- Generate Liquibase by classes; or generate classes by liquibase?
    - Back then possible with JPA
    - Or we keep it in-sync by hand
    - How is it different than API-spec?
    - maybe good to have file for contract, to make more aware if you do changes

Other (semi-related) problem: How to handle if there are MANY changelogs?

- It slows down startup considerably
- Compress/squash them?!

Testing

- All tests are automated; there is not a single manual test!
- We have full confidence in our tests as a safety net.

.1. Test Types

.1.1. Unit Tests

- In OO-languages, the smallest unit is a class (object); tests grouped by its public methods.
- Run extremely fast, easy to write (if code is well designed/testable), stable (low maintenance).
- There is not a single framework (besides mocking if applicable) involved.
- They test business logic code, not integration code (smell: overuse of mocking).

.1.2. Integration Tests

- Narrow definition: Everything testing 2+ classes; broad definition: Testing the application as a whole.
- Never leaves the application boundary, thus in **full control**; stable, but a bit slower.
- Relying on an (implicit/self-defined) contract is risky (better E2E).

.1.3. End-to-End (E2E) Tests

- About:
  - Using the application as a black box; only acting upon a contract (requirements).
  - Located in a separate GIT repo (no code sharing/dependency; be independent, a parallel stream).
- Pros:
  - Verifies the (implicit) contracts
  - Tests the complexity behind configuring the systems working neatly together.
- Cons:
  - Very slow execution
  - Fragile, as depends on all systems to be up and running.
  - Sometimes the functionality is "hidden" and not testable from the outside.
  - No proper control over the (test) data.
- Details:
  - Using Karate test framework, only change base URL to target another environment.
  - Use @WIP tagging; maybe Jira ID @ABC-12345
  - Each environment (DEV, FB-*, TEST, ACC) is running them
    - Could provide @ReadOnly to target also PROD
- external parties need to provide test-environment/sandbox, otherwise mock it
  - mocking external parties MUST always comply with real implementation, otherwise our (self-invented) contract is different from actual real implementation

.1.3.1. E1E Test

- A mix between an Integration Test and an End2end Test is a E1E Test (half of a E2E).

- It works under same assumptions as E2E (blackbox, Karate) but all real systems are mocked.
    - Additionally it has access to system internals through an additionally deployed test API.

.1.4. Other Types

- ATDD, BDD Tests
- performance/load/stress tests
    - We for sure need one of those! Use gatling (with karate?) or jmeter
- Backend Tests / external system tests
- contract tests
- system tests
- acceptance/functional tests

Other:

- application tests?
- module tests?
- component tests?
- smoke tests
- regression tests
- security/penetration tests (outsourced?)

Other other:

- sanity tests
- exploratory tests
- usability tests
- compatibility tests
- localization/internationalization tests

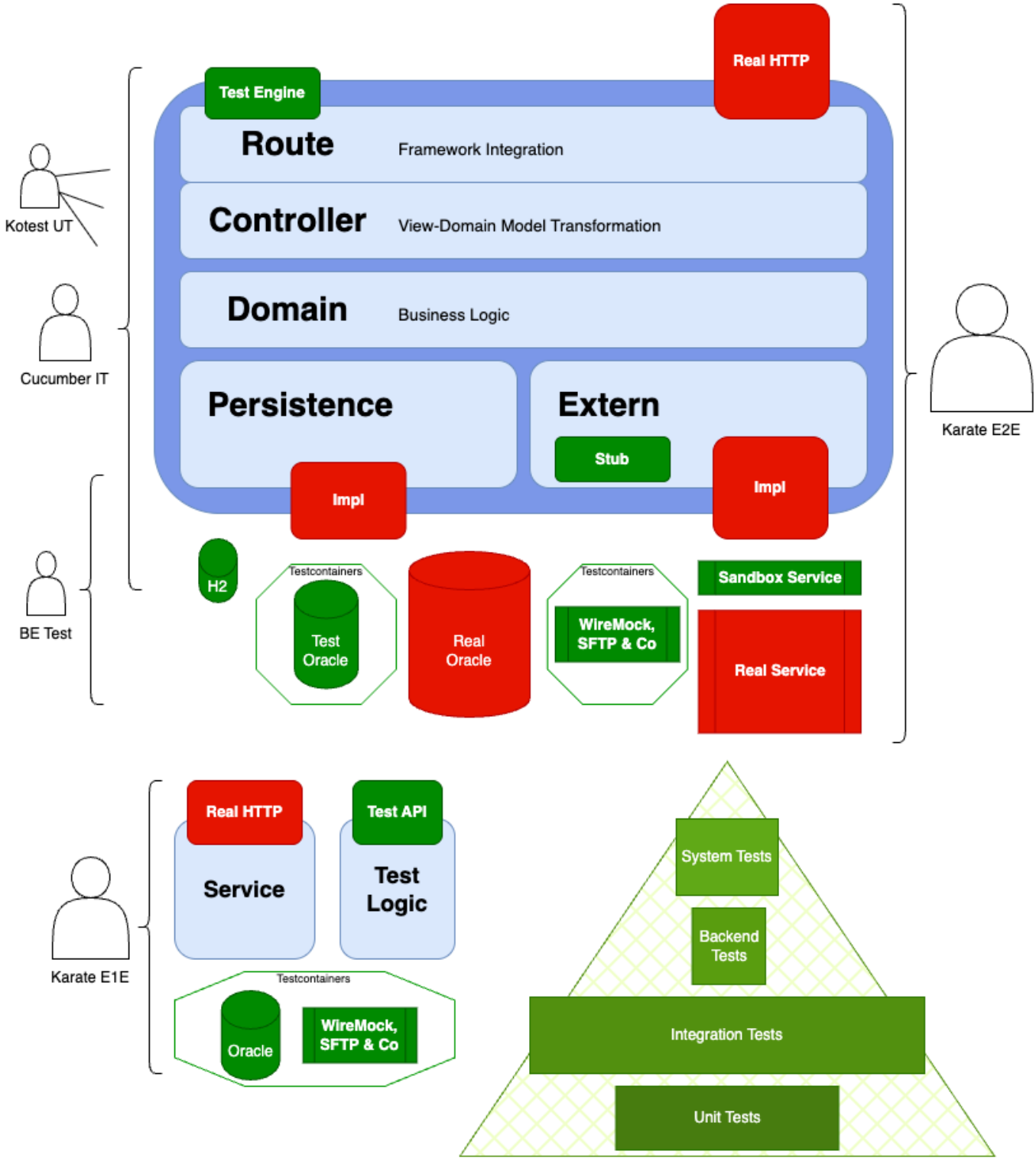

Figure 4. Simplified view from different test strategies onto the application boundaries

- Some Unit Tests (using Kotest framework) covering logic (not infrastructure/IO related code).
- Lots of Integration Tests (using Cucumber framework) verifying functional requirements.
  - Either using fast test engine, or real HTTP (slower, more production-similar).
  - Either using H2 or Oracle in a testcontainer (slower, more production-similar).
  - Has the Koin context available, thus full control of the internals (extern-stub or custom mocks).
  - The parts, the IT don't cover, can be complemented by separate tests.
  - Karate not feasable anymore, due to need of custom vocabulary (step definitions).
- Few End2end Tests talking real HTTP, separately deployed, no access than a regular user

would.
- A sandbox environment of 3rd party services would be great (if available).
    - Do we want to provide one ourselves to others?!

Ad complementing:

- Backend Tests, only testing the extern-layer with testcontainers.
    - Could make them system tests (without testcontainers! sandbox?), but super fragile/limited.

5.1. Best Practices

Watch out to not test same multiple times, we need to be able to still move and change fast!

- Test for HTTP codes (sometimes returning 500 if authorization is not present for the user)
- Think and test blackbox (you don't know the implementation)
    - E.g. just because the same mechanism is applied to several endpoints with a cross cutting concern (security, pagination, …), doesn't justify not testing it.
    - To overcome DRY, write sophisticated test infrastructure (Kotest **`include()`** test factory)
- There also always the possibility to deploy another service exposing additional test endpoints ;)

Watch out for fast test execution:

- Always parallelize all tests; requires isolated state of whole application!
- Use Ktor test-engine for simulated, and much faster, testing HTTP communication.

5.2. Testable Code

- no manual instantiation or static random data generation

5.3. Maintainable Tests

- Assert only what's relevant, thus less in many cases.

5.4. Anti Patterns

- Not keeping proper level of abstraction: Displaying irrelevant values.
- Hardcoded, OS-specific, absolute paths in code or properties.
- Using the filesystem; if really needed, use Java's temp file mechanism.
- Not testing "automagic" (implicit stuff; aspects, proxies; reflection, annotations)
    - Previoiusly Spring repository interfaces (not visible in coverage as well)

5.5. Test Tech

5.5.1. Use BDD

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- TDD on unit test level; BDD for integration tests (on higher acceptance/functional level)
- using a gherkin-based format; use cucumber
- when having both plugins (cucumber and karate) installed, configure file types to be different
    - cucumber: *.feature
    - karate: *.karate.feature

5.5.2. Use Cucumber Lambdas

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- use more kotlin-idiomatic approach by the java8 lambda functionality
    - instead the old fashioned annotationstyle
- both approaches are fully supported by the intellij-plugin
    - click into declaration possible from feature files

Links:

- https://github.com/cucumber/cucumber-jvm/blob/main/cucumber-java8/README.md
- Example: https://github.com/cucumber/cucumber-jvm/blob/main/cucumber-kotlin-java8/src/test/kotlin/io/cucumber/kotlin/LambdaStepDefinitions.kt

5.5.3. Verify Coverage

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- Goal: automatically verify minimum test coverage threshold
    - Fail build if not achieved
    - Provide dashboards (Sonarqube) on current state and changes (delta)
- coverage 100%?, kotlin inlining issue…

Options:

- jacoco
    - very mature, well supported by tools
    - other coverage tools usually can export to jacoco's format too
    - not supporting Kotlin's inline
- kover
    - specific for kotlin, from jetbrains
    - bleeding edge (child sicknesses?)

5.5.4. Assert JSON

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- for itest with cucumber, we need a library which supports something like:
  `$.items[3].title eq 'foobar'`

- a library which we can be passed a complex string to evaluate
- for complete JSON comparison, use: https://github.com/skyscreamer/JSONassert

Options:

- assert full JSON: https://github.com/skyscreamer/JSONassert
- regular expression support: https://fslev.github.io/json-compare/
- jayway path is an option (`com.jayway.jsonpath:json-path`)
- it provides hamcrest matchers `com.jayway.jsonpath:json-path-assert`
- tutorial: https://www.baeldung.com/guide-to-jayway-jsonpath

Chapter 6. Code Quality

- quality gates
  - make the build fail upon error
  - 0-warning policy
- sonarqube (misc metrics), coverage kotlin tools (detekt, ktlint)
- CAVE: there is much overlap in the tools
  - test coverage enforcing by gradle (kover) and sonarqube
  - lots with detekt/ktlint (intellij? sonarqube?)

6.1. Static Code Analysis

Detekt
ktlint

- https://pinterest.github.io/ktlint/1.8.0/

6.2. SonarQube

- use the free tier for open source projects
  - See: https://sonarcloud.io/project/configuration/AutoScan?id=seepick_diamond
- define new code by days (continuous delivery), not by change delta (planned releases)
- use it to report coverage (enforced by gradle-kover itself, not sonar)

6.3. Static Code Analysis

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- actually also runtime detection possible!
- also ktlint
- enable **`Refactor → AutoCorrect by detekt rules`**
- bad: e.g. line length not auto-configured in intellij by detekt rule :-(

6.4. Explicit ExceptionADR - Handling

Status: Draft
Created: 1.12.2025
Author: Seepick

Context:

- Ever since the time when the Spring Framework got popular, unchecked exceptions were favored over checked ones. (Neil Gafter, Josua Bloch)
  - Kotlin goes one step further, and treats all exceptions as unchecked ones.
- Throwing exceptions is a sort of an implicit return value, which can be forgotten to be handled properly (resulting in false-500 responses).

Implicit exceptions are easily forgotten and unhandled

```
fun Service.dangerousMethod(): Result {
    throw RuntimeException()
}
fun Route.handleGet(): ServerResponse {
```

```
    val result = service.dangerousMethod()
    return toResponse(result)
}
```

Proposal:

- Use the Arrow library to introduce a more functional approach, its **`Either`** type specifically.

Explicit fault handling cannot be forgotten nor unhandled

```
fun Service.dangerousMethod(): Either<Failure, Result> {
    // do some real logic ...
    return Failure().left()
}
fun handleGet(): ServerResponse =
    service.dangerousMethod().fold(
        ifLeft = { toFailResponse(it) }, (1)
        ifRight = { toResponse(it) },
    )
```

(1) in order to access the result, we need to unwrap both

- Good:
  - developers need to think (forced to, explicit) error types, mapping of them (translation of corrupt data to bad request/internal error)
  - supports DDD by simulating failure-aware constructors (companion operator fun with Either return type)
- Bad:
  - sometimes not clear if not familiar with it
  - sometimes IDE doesn't suggest **`.bind()`** although it's the solution

Chapter 7. DevOps

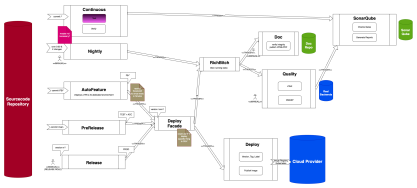

Figure 5. Sketching a build pipeline overview

- When a feature branch is created, a new environment gets auto-created.
    - Testers can write their E2E tests targeting it.
        - Development starts in parallel: The contract (requirements) the base.
- On merge: Old feature-branches are deleted and custom environment destroyed.

7.1. GitHub Actions

- Continuous
- Nightly
- Release

7.2. Distributed Services

Their specifics require:

- Versioning, backwards compatiability.
- Resilience and Fault Tolerance: circuit breakers, retries, and bulkheads.

7.2.1. Versioning Scheme

Status: Draft
Created: 6.12.2025
Author: Seepick

Context:

- Deployed software has to be uniquely identifiable.
- The version number needs to be generated automatically (CI/CD).
- It will be exposed in various places (APIs, UIs, documentation, etc.).

Options:

1. One single continuous version number for each build.
    - The number becomes meaningless, but very simple to implement.
2. Semantic Versioning (SemVer)
    - Well known and widely adopted.
    - Major, Minor, Patch conveys information about backward compatibility.
3. Calendar Versioning (CalVer)
    - Easy to understand and communicate.
    - Conveys information about the release date.

Resources:

- E.g. use GitHub action for that: https://github.com/marketplace/actions/version-increment

Chapter 8. Appendix

8.1. Resources

- ADR: https://github.com/joelparkerhenderson/architecture-decision-record
- SFTP inspirations:
  - https://github.com/alkaphreak/marstech-sftp/blob/main/src/main/kotlin/fr/marstech/mtsftp/service/SftpFileConnectorServiceImpl.kt
  - https://medium.com/whozapp/sftp-test-implem-of-jsch-with-kotlin-testcontainers-and-spring-boot-native-537f624da895
  - https://github.com/testcontainers/testcontainers-java/blob/main/examples/sftp/src/test/java/org/example/SftpContainerTest.java
- Exposed:
  - DAO: https://ktor.io/docs/server-integrate-database.html#create-mapping
- Cucumber:
  - https://www.baeldung.com/java-cucumber-hooks
  - https://towardsdev.com/how-to-perfectly-test-your-ktor-application-94fb92c5a303
  - https://www.baeldung.com/cucumber-data-tables
  - https://github.com/cucumber/cucumber-jvm/tree/main/datatable
- AsciiDoc themes https://gist.github.com/misuo/5b2af22ca78d5d87c522a817a7a8569d